

Relatório técnico: Ajustes para códigos de entrada, perfis e conversas privadas

Visão geral

O aplicativo atual usa Firebase (Firestore + Storage), React (Vite) e serviços próprios para eventos, perfis e conversas. A versão enviada tinha as seguintes limitações:

1. **Códigos de entrada e QR Codes** eram exibidos indiscriminadamente, mesmo para quem não é organizador. Isso permitiria que qualquer usuário se auto-incluisse em um evento sem passar pela confirmação do organizador.
2. **Perfis de participantes e conversas:** a listagem dos participantes era confusa e não havia uma forma clara de entrar em contato com outra pessoa. O ideal é aproximar a experiência de redes sociais, em que cada participante tem um “cartão” com foto e algumas informações básicas, e um botão para iniciar conversa privada.
3. **Conversas privadas:** as conversas não carregavam ou apresentavam erros de assert interna do Firestore. A arquitetura atual misturava cadastro de conversas com lógica de eventos, ficando difícil de depurar.

O relatório a seguir detalha um conjunto de modificações que permitem:

- Controlar quem pode visualizar o código/QR Code de um evento (apenas organizador).
- Organizar a listagem de participantes de modo similar a Facebook/Instagram: fotos de perfil, dados básicos (nome, bio/preferências) e botão “Conversar”.
- Simplificar o fluxo de criação/carregamento de conversas para que sejam sempre privadas e vinculadas aos participantes.
- Corrigir erros de “FIRESTORE internal assertion failed” substituindo lógica de listeners complexos por funções diretas.

1. Controle de visibilidade do código e QR Code

- **Persistência:** os eventos continuam contendo os campos `code`, `entryUrl` (link encurtado) e `checkInEnabled`. Porém, apenas as telas e serviços usados por organizadores deverão exibir esses campos.
- **Serviço de eventos:** em `services/eventService.ts` é recomendado filtrar as propriedades retornadas de acordo com a role do usuário. Um exemplo simples:

```
export async function getEventById(eventId: string): Promise<EventItem | null> {
  const snap = await getDoc(doc(db, "events", eventId));
  if (!snap.exists()) return null;
  const data = snap.data() as EventItem;
  // nunca retorne o código se o usuário logado for comum
  const current = getAuth().currentUser;
  if (!current) return data;
```

```

const profile = await getProfile(current.uid);
if (profile?.role !== 'organizer' && profile?.role !== 'admin') {
  delete (data as any).code;
  delete (data as any).entryUrl;
}
return data;
}

```

- **Organizer dashboard:** a tela `OrganizerDashboardPage.tsx` já é restrita via `ProtectedRoute` aos papéis `organizer` e `admin`. Mantenha ali os botões de copiar/imprimir QR code e habilitar/desabilitar entrada. Para usuários comuns, mova o `EventCard` para uma tela de “Eventos” que não renderize nenhuma informação sensível:

```

{/* EventCard para usuários comuns */}
<article className="glass-panel rounded-[32px] p-6" key={event.id}>
  <div className="flex justify-between items-center">
    <div>
      <h2 className="text-2xl font-semibold text-white">{event.name}</h2>
      <p className="text-sm text-white/60">{event.venueName}</p>
    </div>
    <span className="text-xs font-medium text-accent-pink">{formatEventStatus(event)}</span>
  </div>
  {/* não mostra código */}
  <p className="mt-3 text-sm text-white/70">Entrada liberada:
  {event.checkInEnabled ? 'Sim' : 'Não'}</p>
  {/* Botão para entrar no evento quando check-in estiver habilitado */}
  {event.checkInEnabled && (
    <Link className="mt-4 inline-flex items-center rounded-full px-4 py-2 bg-accent-purple text-white" to={`/entrar/${event.code}`}>Entrar na festa</Link>
  )}
</article>

```

Dessa maneira, o código de entrada fica restrito à rota do organizador. Usuários acessam o evento através de `/entrar/:codigo`, mas não veem o código antes da hora.

2. Listagem de participantes e perfil estilo rede social

Crie um componente para mostrar participantes em lista ou grade. Esse componente deve carregar a lista de `EventParticipant` via o serviço já existente (`subscribeToParticipants`) e buscar os dados de perfil de cada participante usando `getProfile(uid)`.

Exemplo de `ParticipantList.tsx`

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { getProfile } from '@services/profileService';
import { EventParticipant, Profile } from '@types';

```

```

interface Props {
  participants: EventParticipant[];
}

export function ParticipantList({ participants }: Props) {
  const navigate = useNavigate();
  const [profiles, setProfiles] = useState<Record<string, Profile | null>>({});

  useEffect(() => {
    async function loadProfiles() {
      const newProfiles: Record<string, Profile | null> = {};
      await Promise.all(
        participants.map(async (p) => {
          const prof = await getProfile(p.userId);
          newProfiles[p.userId] = prof;
        })
      );
      setProfiles(newProfiles);
    }
    loadProfiles();
  }, [participants]);

  return (
    <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
      {participants.map((p) => {
        const prof = profiles[p.userId];
        if (!prof) return null;
        return (
          <div
            key={p.userId}
            className="rounded-2xl bg-base-800 p-4 flex flex-col items-center hover:bg-base-700 cursor-pointer"
            onClick={() => navigate(`/perfil/${p.userId}`)}
          >
            <img src={prof.photoUrl || '/default-user.png'} alt="Foto de perfil" className="h-24 w-24 rounded-full object-cover" />
            <h3 className="mt-2 text-lg font-semibold text-white">{prof.displayName}</h3>
            <p className="mt-1 text-sm text-white/60 line-clamp-2">{prof.bio}</p>
            <button
              className="mt-3 inline-flex items-center rounded-full bg-accent-purple px-4 py-2 text-white"
              onClick={(e) => {
                e.stopPropagation();
                // abre conversa
                navigate(`/conversas?with=${p.userId}&eventId=${p.eventId}`);
              }}
            >

```

```

        Conversar
      </button>
    </div>
  );
  })}
</div>
);
}

```

Perfil individual

Crie uma página `/perfil/:uid` que mostre a foto, o nome, a bio e um botão “Conversar”. O componente pode buscar o perfil pela rota e chamar `getProfile(uid)`:

```

export function ProfilePage() {
  const { uid } = useParams<{ uid: string }>();
  const { user } = useAuth();
  const [profile, setProfile] = useState<Profile | null>(null);
  const navigate = useNavigate();

  useEffect(() => {
    if (uid) {
      void (async () => {
        setProfile(await getProfile(uid));
      })();
    }
  }, [uid]);

  if (!profile) return <p>Carregando...</p>;
  return (
    <AppShell>
      <div className="max-w-md mx-auto p-6">
        <img className="h-32 w-32 rounded-full mx-auto"
src={profile.photoUrl || '/default-user.png'} alt="Foto" />
        <h1 className="mt-4 text-2xl text-center font-semibold text-white">{profile.displayName}</h1>
        <p className="mt-2 text-center text-white/70">{profile.bio}</p>
        <div className="mt-4 flex justify-center">
          {user?.uid !== uid && (
            <button
className="inline-flex items-center bg-accent-purple text-white px-4 py-2 rounded-full"
onClick={() => navigate(`/conversas?with=${uid}&eventId=${getCurrentEventId()}`)}
>
              Conversar
            </button>
          )}
        </div>
      </div>
    </AppShell>
  );
}

```

```

    </div>
  </AppShell>
);
}

```

3. Estrutura de conversas privada

Modelo de dados no Firestore

A estrutura proposta para conversas privadas é a seguinte:

- Coleção **conversations**: cada documento representa uma conversa entre dois (ou mais) usuários. Campos recomendados:
 - `participants`: `{ [userId]: true }` – dicionário com IDs dos participantes.
 - `lastMessage`: objeto com `text`, `timestamp` e `from` para exibição na lista.
- Subcoleção **messages** em cada documento de conversa: contém documentos de mensagem com:
 - `from`: id do usuário que enviou;
 - `text`: conteúdo da mensagem;
 - `timestamp`: `serverTimestamp()` para ordenar.

Observação: o `eventId` não precisa ficar na conversa; para restringir visibilidade, basta que ambos os usuários estejam presentes no evento. O serviço já verifica via `isUserVisibleInEvent`.

Serviço `chatService.ts` revisado

```

import { collection, addDoc, getDocs, query, where, doc, setDoc,
serverTimestamp, orderBy, limit } from 'firebase/firestore';
import { db } from '../firebase';

// Busca ou cria uma conversa entre dois usuários
export async function getOrCreateConversation(user1: string, user2: string) {
  const convQuery = query(
    collection(db, 'conversations'),
    where(`participants.${user1}`, '==', true),
    where(`participants.${user2}`, '==', true),
    limit(1),
  );
  const snapshot = await getDocs(convQuery);
  if (!snapshot.empty) {
    return { id: snapshot.docs[0].id, ...snapshot.docs[0].data() } as
Conversation;
  }
  const docRef = doc(collection(db, 'conversations')); // gera id
  await setDoc(docRef, {
    participants: { [user1]: true, [user2]: true },
    lastMessage: null,

```

```

    });
    return { id: docRef.id, participants: { [user1]: true, [user2]: true },
lastMessage: null };
}

// lista todas as conversas de um usuário e carrega dados do outro
participante e da última mensagem
export async function listUserConversations(userId: string):
Promise<Conversation[]> {
    const qs = query(collection(db, 'conversations'), where(`participants.$
{userId}`, '==', true));
    const snapshot = await getDocs(qs);
    const convs: Conversation[] = [];
    snapshot.forEach((doc) => convs.push({ id: doc.id, ...(doc.data() as
any) }));
    return convs;
}

export async function sendMessage(conversationId: string, from: string,
text: string) {
    const msgRef = await addDoc(collection(db, 'conversations',
conversationId, 'messages'), {
        from,
        text,
        timestamp: serverTimestamp(),
    });
    // atualiza lastMessage
    await setDoc(
        doc(db, 'conversations', conversationId),
        {
            lastMessage: { from, text, timestamp: serverTimestamp() },
        },
        { merge: true },
    );
    return msgRef;
}

export function subscribeToMessages(
    conversationId: string,
    onUpdate: (msgs: Message[]) => void,
) {
    const q = query(
        collection(db, 'conversations', conversationId, 'messages'),
        orderBy('timestamp', 'asc'),
    );
    return onSnapshot(q, (snapshot) => {
        const messages: Message[] = [];
        snapshot.forEach((doc) => messages.push({ id: doc.id, ...(doc.data() as
Message) }));
        onUpdate(messages);
    });
}

```

```
});
}
```

Essa implementação elimina o uso de um aggregator complexo (causador do erro de assert) e usa consultas simples com `where`. O uso de `orderBy('timestamp', 'asc')` no listener evita estados inconsistentes.

Página de conversas revisada

O componente `ConversationsPage.tsx` precisa separar três responsabilidades:

1. **Listar conversas existentes** do usuário (com nomes e fotos dos outros participantes e a última mensagem).
2. **Criar/carregar uma conversa** quando o parâmetro `with` estiver presente na URL. Isso requer validar se ambos os usuários estão visíveis no evento antes de criar.
3. **Exibir o painel de chat** (mensagens) quando uma conversa for selecionada.

Abaixo segue um esboço de como isso pode ser feito, reaproveitando o serviço acima e o componente `ParticipantList` para abrir as conversas:

```
export function ConversationsPage() {
  const { user } = useAuth();
  const [searchParams] = useSearchParams();
  const [conversations, setConversations] = useState<Conversation[]>([]);
  const [selectedConversation, setSelectedConversation] =
    useState<Conversation | null>(null);
  const [messages, setMessages] = useState<Message[]>([]);

  const eventId = useMemo(() => searchParams.get('eventId') ??
    getCurrentEventId(), [searchParams]);
  const targetUserId = searchParams.get('with');

  // carrega lista de conversas do usuário
  useEffect(() => {
    if (!user) return;
    let active = true;
    void (async () => {
      const convs = await listUserConversations(user.uid);
      if (active) setConversations(convs);
    })();
    return () => {
      active = false;
    };
  }, [user?.uid]);

  // cria ou abre conversa quando ?with=xyz
  useEffect(() => {
    if (!user || !targetUserId) return;
    let sub: Unsubscribe | null = null;
    void (async () => {
```

```

// verifica se ambos estão visíveis no evento
const [selfVisible, targetVisible] = await Promise.all([
  isVisibleInEvent(eventId, user.uid),
  isVisibleInEvent(eventId, targetUserId),
]);
if (!selfVisible || !targetVisible) return;
const conv = await getOrCreateConversation(user.uid, targetUserId);
setSelectedConversation(conv);
sub = subscribeToMessages(conv.id, setMessages);
})();
return () => {
  if (sub) sub();
};
}, [targetUserId, user?.uid, eventId]));

// renderização
return (
  <AppShell>
    <div className="grid grid-cols-[1fr_2fr] gap-6">
      /* Lista de conversas */
      <section className="rounded-2xl bg-base-800 p-4">
        <h2 className="mb-4 text-lg font-semibold text-white">Conversas</
h2>
        {conversations.map((conv) => {
          const otherId = Object.keys(conv.participants).find((id) => id !
== user?.uid);
          return (
            <ConversationListItem
              key={conv.id}
              conversation={conv}
              otherUserId={otherId!}
              onClick={() => setSelectedConversation(conv)}
            />
          );
        })}
      </section>
      /* Painel de chat */
      <section className="rounded-2xl bg-base-800 p-4">
        {selectedConversation ? (
          <ChatPanel
            conversationId={selectedConversation.id}
            messages={messages}
            onSend={(text) => sendMessage(selectedConversation.id,
user!.uid, text)}
          />
        ) : (
          <p className="text-white/60">Selecione uma conversa ou escolha
alguém na lista para começar.</p>
        )}
      </section>
    </div>

```



```

    </AppShell>
  );
}

```

`ConversationListItem` busca os dados do outro participante através de `getProfile(otherUserId)` e exibe foto, nome e última mensagem (campo `lastMessage`), facilitando a experiência parecida com WhatsApp.

`ChatPanel` deve aceitar as mensagens via props e renderizá-las em ordem, além de possuir um campo de entrada para envio.

4. Ajustes adicionais e como testar

1. **Dependências:** instale a biblioteca `react-qr-code` conforme já incluído no `package.json` do front. Rode `npm install` dentro de `client`.
2. **Roteamento:** adicione rotas em `App.tsx` para:
 3. `/perfil/:uid` → `ProfilePage`;
 4. `/conversas` → `ConversationsPage`;
 5. `/entrar/:codigo` → `CheckInPage` (já existe, mas atualize para não exibir código);
 6. proteger `/organizador` com `ProtectedRoute` exigindo role `organizer|admin`.
7. **Testes sugeridos** (conforme pedido):
 8. Transformar um usuário em `organizer` via Firestore (campo `role: organizer`).
 9. Fazer login como organizador, criar evento, copiar o link/código e verificar que o QR Code aparece apenas nessa tela.
 10. Acessar `/entrar/:codigo` sem estar logado: app redireciona para login e depois volta ao check-in; ao confirmar presença, o participante é adicionado em `eventParticipants/<eventId>`.
 11. Fazer login com outro usuário e repetir check-in; verificar que ambos aparecem na lista de participantes no painel do organizador.
 12. Abrir a lista de participantes como usuário comum (ex.: rota `/eventos/:id`); confirmar que as fotos aparecem em forma de cartões com botão “Conversar”.
 13. Clicar em “Conversar” em um participante: conversas devem abrir na rota `/conversas?with=<uid>&eventId=<id>`; enviar mensagens e garantir que aparecem para o outro usuário em tempo real.
 14. Verificar que o erro `FIRESTORE internal assertion failed` não ocorre mais, graças ao uso de `orderBy` e `query` simples.

Com essas alterações, o aplicativo ficará mais próximo do comportamento desejado: perfis visíveis no evento (estilo rede social), chat privado funcional, e controle do código de entrada restrito ao organizador.